OxCidation Shared Documentation

This folder explains the experiment as it exists today. It is intended for colleagues who need to understand the research process, follow development, or inspect a benchmark result without first reading the source code or meeting history.

The Project in One Paragraph

OxCidation compares prompts for translating accepted C programs into Rust with one fixed language model. A generated visible-test suite can identify problems and guide repairs. A separate Judge then evaluates the initial and final Rust programs but never participates in repair. The project already implements this pipeline and its reporting. The remaining work is mainly experimental design: choosing the model, sampling programs, defining prompt-selection criteria, and obtaining suitable Judge cases.

Where to Start

Read according to the question you are trying to answer:

Question	Document
What is the project trying to measure?	Project overview
What happens to one C program?	Pipeline and agents
How will prompts and programs be compared?	Benchmark methodology
What does the Judge actually test?	Judge coverage
Where are results and what do the spreadsheet fields mean?	Outputs and workbook
What must still be decided?	Open decisions
What instructions are sent to the agents' LLM?	Agent prompts
Which unexpected cases have already been observed?	Unexpected smoke-test cases
What happens to one program in the main experiment?	Simplified main experiment flow
How does the prompt-selection benchmark work in detail?	Detailed benchmark flow

The repository README contains installation instructions and complete command examples.

Key Terms

Term	Meaning in this project
Problem	A programming task with one specification and one Judge suite.
Program or snippet	One accepted C submission that solves a problem.
Prompt	The instruction template used to ask the model for a Rust translation.
Visible tests	Generated inputs whose failures may guide repair.

Judge	External execution evidence that is recorded but never used for repair.
Result	One C program translated with one prompt and one fixed model.

Current Position

The project is between a working research prototype and the definitive experiment.

Working now

- Extraction of accepted C programs from CodeNet.
- Reproducible source and prompt selection.
- Translation, compilation, generated visible tests, semantic validation, and bounded repair.
- Deterministic visible-input filtering, per-case Validator review, one bounded replacement round, and frozen suite reuse across prompts.
- Operational generation or review failures remain separate from semantic inconclusive decisions and are retried on benchmark resume.
- Independent initial and final Judge observations.
- Resumable exact description-hash mapping from accepted-C CodeNet AIZU problems to AOJ IDs.
- Resumable mapped AOJ system-suite retrieval with explicit external IDs.
- Multi-prompt execution with a fixed model.
- Resumable, isolated runs with CSV, JSON, logs, code versions, and an Excel-compatible analysis workbook.

Working, but not methodologically complete

- Visible-case review is model-based and cannot confirm constraints absent from the C source; cases without supported validity are discarded and suites with no approved cases are disabled and reported instead of influencing Rust repair.
- Exact AIZU mapping and system-suite prefetch are complete. The remaining eligibility step is a population-wide accepted-C baseline validation. The earlier numeric-mapping corpus is invalid for research use. AtCoder system coverage remains unavailable.
- The workbook is useful for automatic analysis and navigating artifacts. The need and format for any separate manual inspection protocol remain open.

Still to be fixed or decided

- Definitive model and provider settings.
- Number of problems and programs per problem.
- Separate prompt-selection and final-evaluation samples.
- Primary metric for choosing a prompt.
- Population-wide accepted-C baseline validation for the mapped AOJ suites.
- Treatment of unavailable Judge cases and special judges.

Documentation Scope

The shared folder should remain an overview, not an archive. Detailed meeting records, historical alternatives, and implementation investigations remain in `docs-local/`. When implementation or methodology changes, this folder should be updated to describe the new current state.